

AI-Wrevolusi

System Architecture Report

FIT5120 Industry Experience | Iteration 2 | 17 September 2026

Current code baseline: aa4f8fb on kuan/design-prototype

Purpose and main outcome

AI-Wrevolusi helps users understand how AI may affect their work and choose practical learning steps. This report explains how our architecture has changed from Iteration 1 to the current version. It covers the frontend, backend, database, AI integrations and deployment setup.

The main structure is still a React frontend, a FastAPI backend and a PostgreSQL database. The larger changes are in how we save user work, share learning data between pages and check that the system behaves as expected. The latest refactor keeps the page-based interface and gives shared profile and learning operations clear owners.

A key change is that editing a work profile no longer clears saved courses or calendar entries. We keep the work and ask the user to review whether the recommendations and plan still fit. Recording task practice does not trigger that review.

What this report compares

Baseline	What it tells us
Iteration 1 design	The original intended data flow and ERD in docs/iteration1_erd.md.
Earlier implementation	Verified release and branch snapshots, including the state before this refactor at 818cfae.
Current implementation	The committed source and local validation at aa4f8fb. Live deployment and database verification remain separate.

The report uses code and Git history to explain what changed. Reasons are tied to concrete problems and trade-offs rather than assumed team discussions. Source references are listed at the end. [S1-S10]

1. How the architecture changed

The architecture developed in stages. Learning and account features grew during Iteration 2. The latest work then corrected shared ownership and saving behavior without replacing the whole application.

Stage	Evidence	Main change
Initial design 28 August	ff57c98	E1-E4 design with raw, reference and business data. Learning and planning were outside the original ERD.
Iteration 1 implementation 2-3 September	3801a0a release 2499736	Work profile and exposure features. The frontend saved profile work in the browser. A shared API client and Vercel service split already existed.
Account and learning growth 7-16 September	853d687 36b2a09 8fac476	Account workspace storage, verified courses, chapter progress, check-ins and daily brief records.
Setup improvements 16 September	9ae7062 c941f7b	UV lockfile and dependency groups, followed by one localhost launcher.
Frontend layout changes 16 September	0e70bac 8c27a04	A broad features layout was added, then code was consolidated under pages. The old architecture guide and check commands became inconsistent.
Immediate review baseline 17 September	818cfae	Plan and Learning Centre shared each other's state logic. Profile writes could clear saved learning.
Current bounded refactor 17 September	aa4f8fb	Keep page UI, extract shared state operations, preserve saved work and restore executable checks.

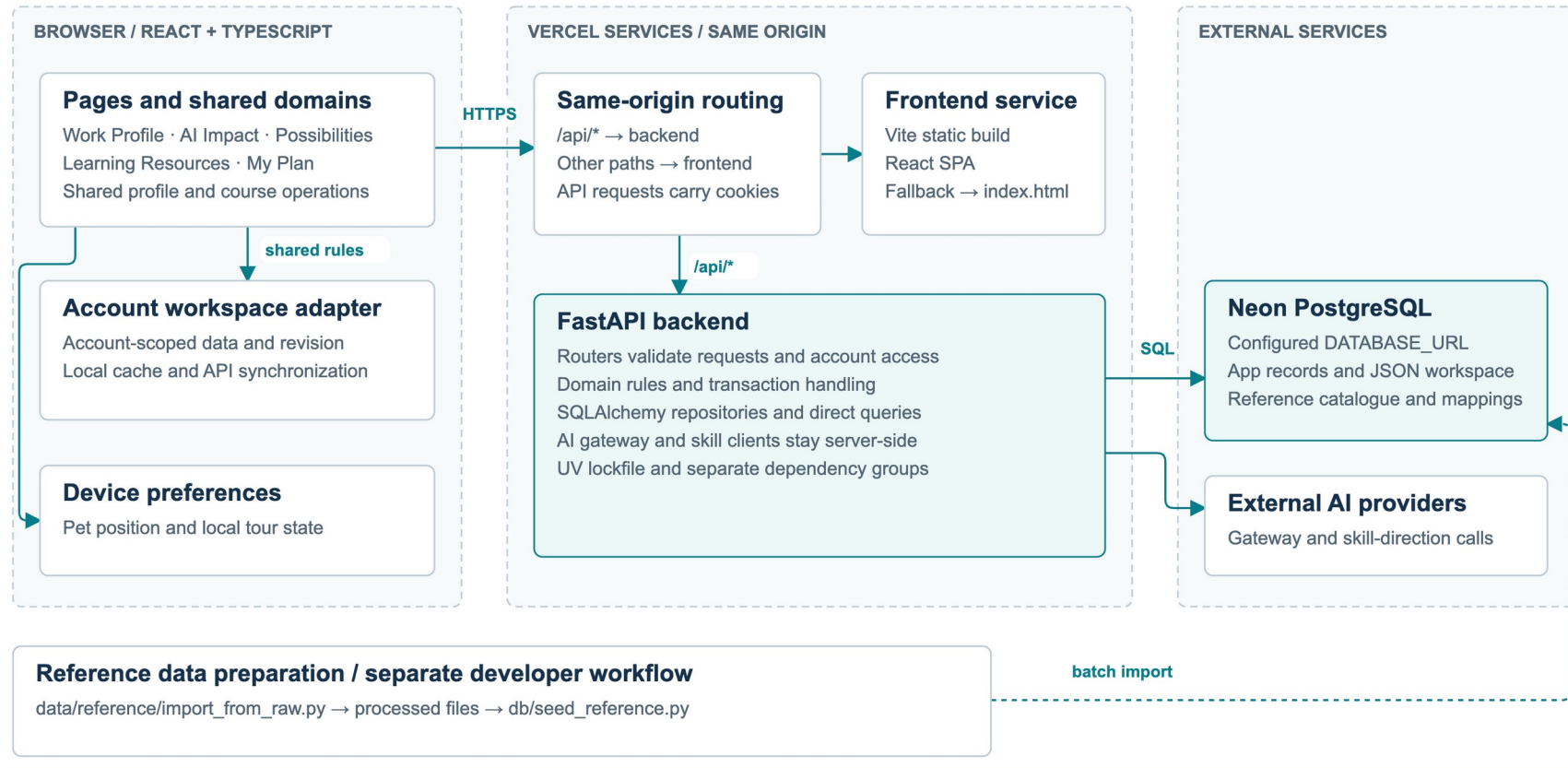
The named iteration-1-release tag points to 2499736 on a different line of Git history. It is a valid release snapshot, but it is not a direct ancestor of the current branch. Commit 3801a0a is an earlier Iteration 1 milestone on the current branch history. [S1, S2]

2. Current system architecture

AI-WREVOLUSI / ITERATION 2

System architecture

Code and configuration view at aa4f8fb • No live deployment or database verification



Arrows show request or dependency direction. The dotted path shows the separate offline data import workflow.
Local development uses ./dev and the Vite API proxy. Hosted routing shown from vercel.json, not verified production state.

Figure 1. Current source and configuration view at aa4f8fb. The browser talks to the backend API. Database access and external AI calls stay on the server side. Batch data preparation is separate from user requests. [S3, S6, S9]

3. Frontend changes and reasoning

Our route pages still own their screens, forms and page-only components. We moved shared profile and learning state into two small domain groups. Plan and Learning Centre now call the same course operation instead of importing each other's storage logic. [S4]

Before the refactor	Current behavior	Why it helps
Plan read Learning Centre internals, while the library hook called Plan synchronization.	Shared learning-planning modules own course contracts, the library and course add/remove operations.	Both pages use one rule for the same saved course.
A changed task or analysis cleared learning records, including some practice-only updates.	Structural changes preserve records and set a review flag. Practice-only updates preserve records without that flag.	Users do not lose saved work when they record a trial or change their profile.
Plan read and wrote an unscoped browser copy even when signed in.	Signed-in work uses the account workspace. Guest import is explicit and legacy imports stay in the guest workspace.	An old browser copy cannot silently become another account's plan.
The guide described missing folders and check commands.	A restored check command resolves imports and checks the boundaries used by the code.	The team can test the documented rules rather than rely on folder names.

What stays in the pages

Cards, drawers, calendars and page hooks remain close to their screens. Shared evidence components under Analysis and Skills can still be composed by other pages. We did not move all of this UI because reuse alone is not a harmful dependency. Shared modules and generic UI, however, cannot depend on page internals.

Account data and device preferences

The account storage service owns synchronized keys, account-specific caches and revision checks. A small local-preference adapter stores things such as pet position and tour memory. Those preferences do not need to become account business records. [S4, S5]

4. Backend changes and reasoning

FastAPI still runs as one application with explicit routers. Services hold business rules and persistence operations, while repositories are used where they already provide a useful boundary. We focused on who validates and commits data rather than adding a repository class for every table. [S6]

Issue in the previous code	Current change	Practical reason
A task repository committed before the service calculated exposure.	The task service commits the task and its derived exposure together.	A failed calculation should not leave a partly completed save.
Learning progress used a read-then-write comparison. Duplicate chapter keys could reach a unique constraint.	Duplicate keys are rejected before persistence. A conditional PostgreSQL upsert accepts only a higher stored value.	Retries and overlapping requests should not reduce progress or create duplicate rows.
Check-ins and daily briefs checked for an existing row before inserting.	Check-ins use conflict-ignore. Daily briefs use an atomic upsert.	Two requests for the same key should not race into a duplicate insert.
Workspace writes accepted valid JSON that the Possibilities reader could reject.	Shortlist writes validate their shape. Reads tolerate invalid or retired legacy IDs.	An accepted workspace value should not make a later page request fail.

The boundary is still practical

Some account and Possibilities queries still live in routers. We can extract them when their complexity makes that worthwhile. Moving them only to make every folder look identical would add change without fixing the main issue. The current change makes transaction and validation ownership clearer first.

The new SQL statements and response contracts have offline tests. Real PostgreSQL concurrency and rollback tests are still needed before we claim the behavior has been verified in a live database. [S6, S10]

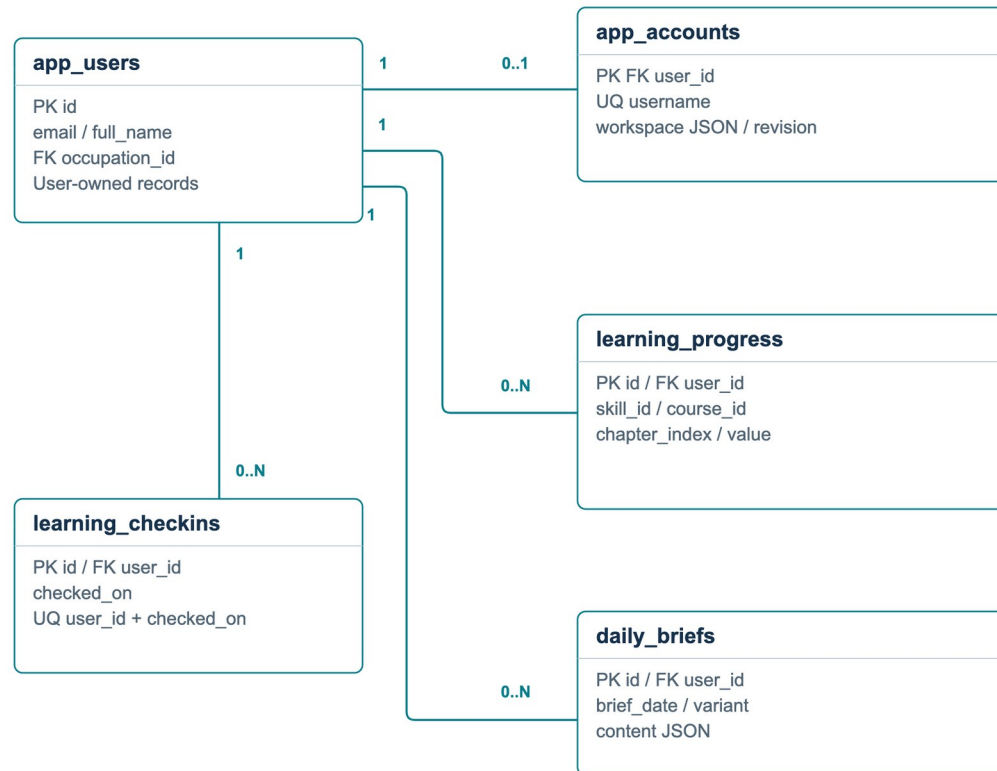
5. Current data model overview

AI-WREVOLUSI / ITERATION 2

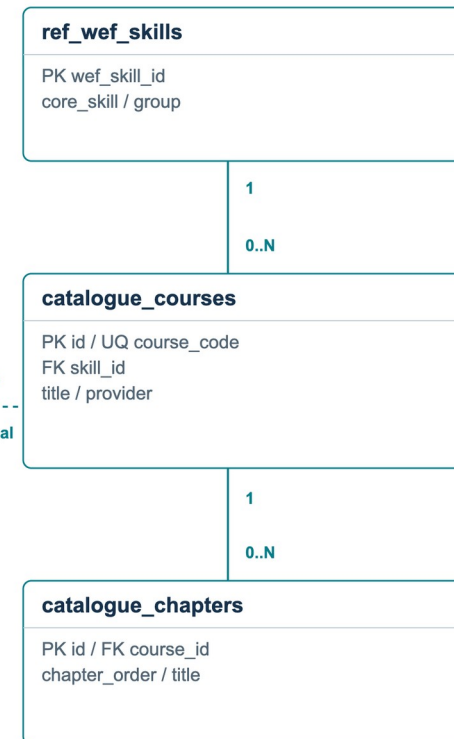
Entity relationship overview

Current core storage at aa4f8fb • Selected key relationships only • Full 26-table source model appears in ERD.PNG

Identity and account workspace



Learning reference catalogue



PK = primary key FK = declared foreign key UQ = unique Solid = declared FK Dotted = logical association only
JSON workspace holds profile, saved learning and plan payloads. They are not separate relational tables in this schema.

Figure 2. Main current relationships. This overview is supported by the full ERD.PNG and the detail diagrams in the appendices. Solid lines show declared foreign keys. Dotted links show application-level associations. Workspace payloads are JSON values rather than separate relational tables. [S7]

6. How the data model changed

The current source defines 15 ORM tables and three SQL reference tables. Eight additional business tables remain in the older SQL schema and are shown separately. This is a source inventory, not a count taken from the deployed database. [S7]

Area	Earlier evidence	Current model
Identity and saved work	The original design showed users and work_profiles. The Iteration 1 frontend stored profile work locally.	app_users supports authentication. app_accounts adds a username, workspace JSON and a revision number.
Learning catalogue	Learning and planning were excluded from the original E1-E4 ERD.	catalogue_courses and catalogue_chapters store the verified course structure.
Learning activity	No corresponding learning activity tables in the release ORM snapshot.	learning_progress, learning_checkins and daily_briefs record learning activity and briefing content.
Task assistance	No task_assist_interactions table in the release ORM snapshot.	One stored interaction is scoped to a user and a browser profile task key.
Latest refactor	The seven new ORM tables had already arrived during Iteration 2.	aa4f8fb changes ownership, validation and writes. It adds no table or migration.

Relationships that need careful labels

app_accounts.user_id is both its primary key and a foreign key to app_users. A user can have no username account row or one account row. Catalogue chapters use a real foreign key to their course, and catalogue courses use a real foreign key to the WEF reference skill.

Learning progress stores a course code, a skill slug and a zero-based chapter index. These values are checked against the catalogue in the application. They are not foreign keys to catalogue UUIDs. The task-assistance task_key is also not a foreign key to tasks.id. [S7]

The original ERD is useful as a design record, but its raw CSV shapes are not current PostgreSQL tables. Its users/work_profiles model also differs from the app_users/account workspace used by the application. We should not describe every difference as a later database migration.

7. Data ownership and request flows

Two forms of saved user data

The account workspace stores the frontend journey as a map of keys to JSON-encoded values. This includes the work profile, saved courses, local course plan and calendar events. Separate relational tables store server-side learning progress, check-ins and daily briefs. The ERD shows both because neither view on its own describes the current system. [S5, S7]

Saving a course or changing a profile

- The user adds or removes a course in Learning Resources, or removes a course in My Plan.
- The shared operation captures the current account session and reads the latest library and plan.
- Related course records are updated together. A failed browser write does not report partial success.
- The account service sends the workspace with its owner and revision. The backend checks the owner, locks the account row and rejects a stale revision.
- If the account changes while catalogue data or a save is pending, the old operation cannot change the new account.

A structural profile change preserves saved courses, progress and calendar entries. The review notice asks the user to check whether they still fit. Mark reviewed only acknowledges the change. It does not regenerate or replace the plan. Explicitly removing a course still removes it from the local plan. [S4, S5]

Reference data has a different lifecycle

MASCO occupations, ILO task scores and WEF skills are prepared through repository data tools and loaded into reference tables. Course catalogue imports are also separate from user requests. The frontend reads these sources through the API. It does not directly update the reference database. MASCO and ILO use an occupation-code association, while WEF skill matching is not a direct four-digit occupation join. [S2, S7]

Why we keep this distinction

A reference refresh and a user save have different owners. Keeping them separate helps us avoid treating a new recommendation as permission to delete the user's choices. It also makes it clearer which data is a source lookup, a calculated suggestion or a saved decision.

8. Deployment, AI and local setup

The deployment structure stays

The repository still configures one Vercel project with a frontend service and a FastAPI backend service. Requests under /api go to the backend first. Other paths go to the frontend, which has a single-page application fallback. Neon PostgreSQL remains an external database. This service split already existed in the Iteration 1 release snapshot, so it is not a new result of the latest refactor. [S3]

Local and deployed paths stay consistent

The browser API client uses /api/v1. During local development, Vite proxies /api to the selected backend port. The ./dev launcher chooses available ports, passes the selected addresses to both services and stops any child process that started if the other one fails. Locked UV execution prevents startup from silently resolving a changed dependency definition. [S8]

AI calls remain a backend responsibility

The initial design proposed NLP followed by AI. The Iteration 1 exposure pilot instead used TF-IDF matching and returned insufficient_data when a match was unreliable. During Iteration 2, the gateway gained configurable providers, Responses API support and a keyless fallback. [S1, S2, S9]

The structured AI endpoints use a gateway with response validation, candidate constraints, retries and fallback logic. Skill directions also have a separate SKILL_LLM provider path. Provider keys belong in backend configuration rather than browser code. [S9]

Missing API credentials do not guarantee offline behavior. The current settings enable a keyless OpenCode fallback by default. This corrects an older description that suggested credential-free operation was always deterministic and local. External provider availability and output quality still need their own checks.

Dependencies and security

Earlier requirements mixed unpinned runtime and development packages. The lockfile and separate groups make setup more repeatable. Python is now limited to 3.12 and uses a committed UV lockfile. Runtime, development, database tooling and data-processing dependencies have separate groups. npm and package-lock.json remain the frontend dependency setup. The documented frontend target is Node 24. [S8]

The code uses authentication cookies, account ownership checks and workspace revisions. Deployment configuration must still set the correct secrets, secure-cookie options and environment-specific database connection. These controls are visible in source, but their hosted values were not verified by this local change. [S3, S5]

9. Why we chose a bounded refactor

We compared two frontend approaches. Both can work. The decision depends on the amount of shared behavior and the cost of changing existing work, not on one folder name being better than another.

Approach	Benefit	Cost and limit
Keep pages and extract shared domains	Moves the shared profile and learning rules to clear owners. The same operations can be tested without mounting a whole page.	Some older evidence components still sit under page folders. Those boundaries need to stay documented.
Restore a broad features and infrastructure layout	Gives more reusable domains a consistent location and may help if many new consumers appear.	Touches more imports and test paths. Moving files alone does not fix data loss, validation or transaction ownership.

We chose the smaller extraction because the confirmed problems were concentrated in shared profile and learning state. This lets us keep the route UI and existing storage formats while fixing the behavior that could lose work. It should also reduce the amount of overlap with other page changes. We have not measured a reduction in merge conflicts, so that remains an expected benefit rather than a measured result. [S1, S4]

Trade-offs we are keeping visible

Workspace JSON makes it practical to save an evolving frontend journey, but it gives the database less control over the shape of each value. We have added validation for the shortlist that the backend reads. Other server-consumed workspace fields may need their own contracts as the application grows.

The frontend still loads its pages eagerly, and the build reports a large JavaScript chunk. Route splitting may help, but it should follow load measurements. The API remains one FastAPI application. There is no current need to split each feature into a separate deployed service.

10. Validation and remaining work

Check	Result	What the result covers
Frontend check	Passed	72 Node tests, resolved boundary checks, lint, TypeScript and the production build.
Focused backend tests	86 passed	Persistence statement contracts, task commit ownership, account validation and learning rules.
Broader guarded backend run	225 passed 9 failed	The same nine failures were reproduced on the starting code. They remain open.
Launcher tests	6 passed	Mocked port selection, shutdown and partial-start cleanup.
Mocked browser check	Passed	Review notice, acknowledgement after reload, course add/remove across both views and retained progress.

These checks give evidence for the changed local behavior. The browser used an in-memory API, and the backend run blocked database and network connections. The results do not prove hosted authentication, live provider behavior or PostgreSQL concurrency. Frontend checks ran on available Node 26.6.0 rather than the documented Node 24. Lint and bundle-size warnings remain. [S10]

Database baseline is still a separate task

The migration chain starts with learning tables and expects `app_users` to exist already. The catalogue migration also expects the reference skill table. Startup `create_all` is a separate schema-creation path, not a migration system. The current ORM metadata also lacks the referenced `ref_wef_skills` table, so an offline metadata check cannot resolve the catalogue foreign key. It is not a complete bootstrap path. We have documented a baseline proposal rather than guessing the state of an existing database.

The next database step is to inspect the approved environment history, then test both an empty database and representative existing schemas in a disposable PostgreSQL environment. Existing migration identities must be preserved. No database migration, stamp or data change was carried out in this refactor. [S7, S10]

Other open work

The nine baseline backend failures cover AI output limits, fallback/evidence expectations and task-assistance tests. A separate `TaskRead/profile_task_id` response-contract mismatch also needs review. These should be tracked separately rather than hidden by changing unrelated tests.

References and evidence

Repository references use the current code baseline aa4f8fb unless an earlier ref is shown. The report text and full data dictionary are included beside the Word and PDF copies. Earlier code can be checked using the Git references listed below.

ID	Source	Use in this report
S1	Git history ff57c98, 3801a0a, 2499736, 0e70bac, 8c27a04, 818cfae, aa4f8fb	Dates, release snapshots and frontend ownership changes. The named Git references preserve the earlier source versions.
S2	docs/iteration1_erd.md db/schema.sql	Original E1-E4 design, reference data and older business schema.
S3	vercel.json backend/main.py docs/deployment.md	Service routing, endpoint and environment boundary.
S4	frontend/ARCHITECTURE.md frontend/src/features/ frontend/scripts/	Shared domain ownership, profile preservation and boundary checks.
S5	frontend/src/services/accountStorage.ts backend/app/routers/accounts.py	Account workspace, revisions, session checks and saves.
S6	backend/app/services/learning_records.py backend/app/services/tasks.py backend/app/services/workspace.py	Atomic writes, commit ownership and shortlist validation.
S7	backend/app/models/ backend/alembic/versions/ Data-Dictionary.md	Current tables, constraints, logical links and migration limits.
S8	backend/pyproject.toml and uv.lock frontend/package.json dev and scripts/dev.py	Dependency management and local startup.
S9	backend/app/core/config.py backend/app/services/ai_gateway.py backend/app/services/skill_directions.py	Configured providers, fallback and separate skill-direction path.
S10	docs/architecture-validation.md docs/database-baseline-proposal.md	Checks run before commit aa4f8fb, known failures and pending database work.

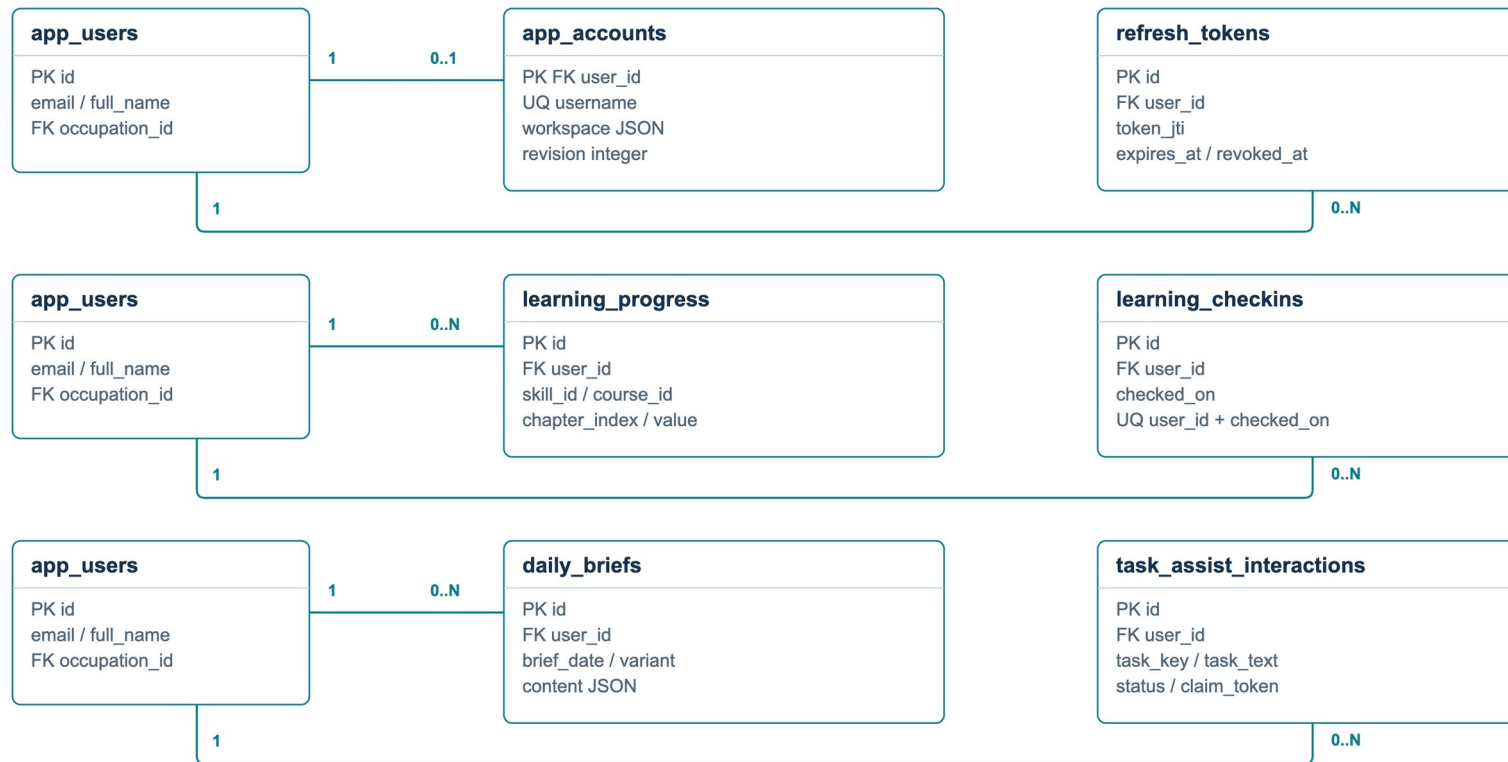
Reference examples: TM04-FutureStack, ParkiCare Iteration 2 System Architecture Report and its two diagrams. TransitReach KL, System Architecture, Iteration 2 working baseline v1.0, dated 17 September 2026. These examples informed the report structure and visual presentation. Their technologies, entities and project claims are not used as AI-Wrevolusi evidence.

Appendix A. Identity and learning records

AI-WREVOLUSI / ITERATION 2

Identity and learning records

Current ORM relationships at aa4f8fb • Repeated app_users boxes are the same table • No live database check



Workspace stores JSON strings by key. task_key identifies a browser task, not tasks.id.

PK = primary key FK = declared foreign key UQ = unique Solid = declared FK Dotted = logical association only
UQ learning_progress = user_id + skill_id + course_id + chapter_index. UQ daily_briefs = user_id + brief_date + variant.

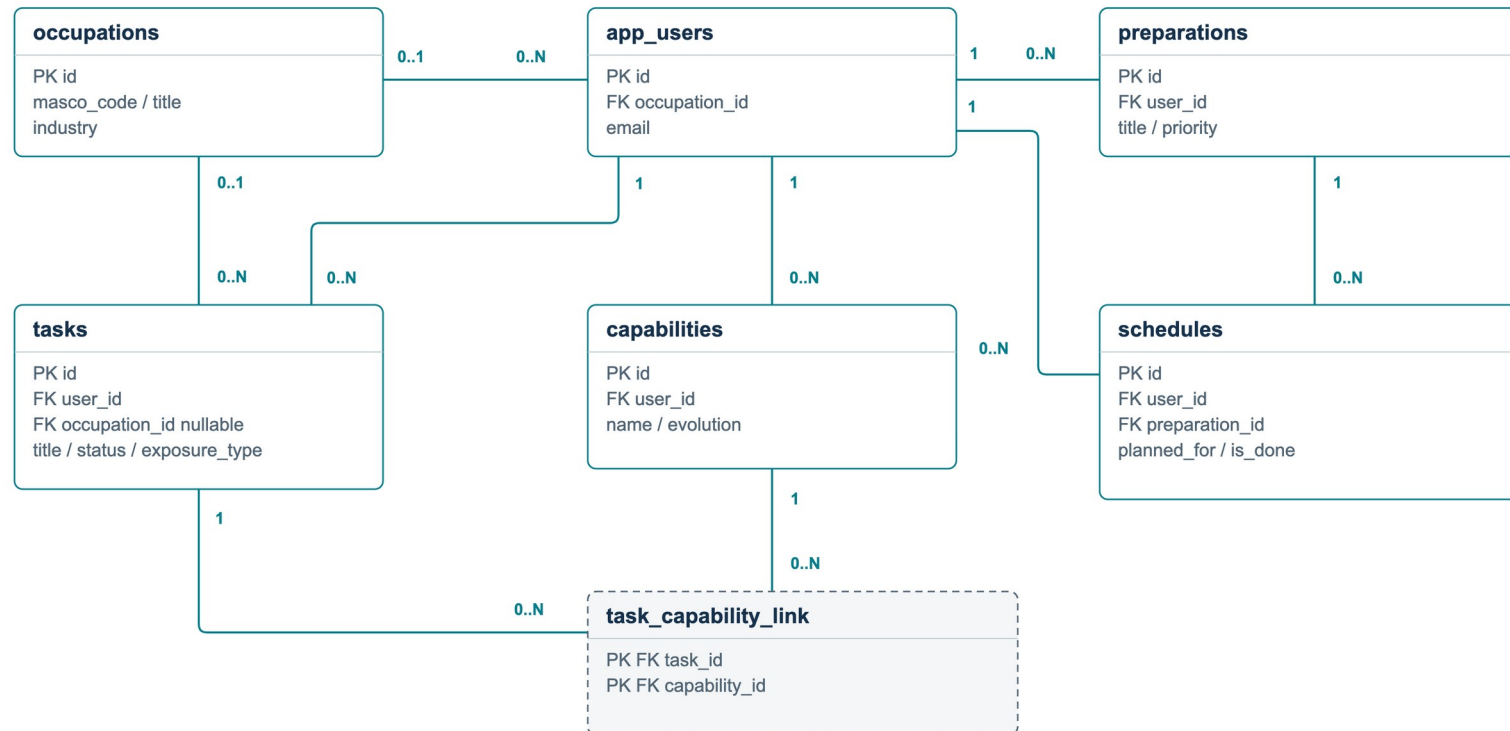
Identity, saved account workspace and learning activity. The diagram shows the declared keys and constraints. Workspace JSON contains frontend journey records, while learning activity also has separate relational tables. [S7]

Appendix B. Work and planning models

AI-WREVOLUSI / ITERATION 2

Work and planning records

Current ORM at aa4f8fb • Dashed table is modelled without an explicit current runtime write • No live database check



The ORM occupations table and reference ref_occupations table are distinct. Their codes are not joined by a declared FK.

PK = primary key FK = declared foreign key UQ = unique Solid = declared FK Dotted = logical association only
All solid links are declared in ORM source. Optional occupation links use SET NULL. Other shown FK links use CASCADE.

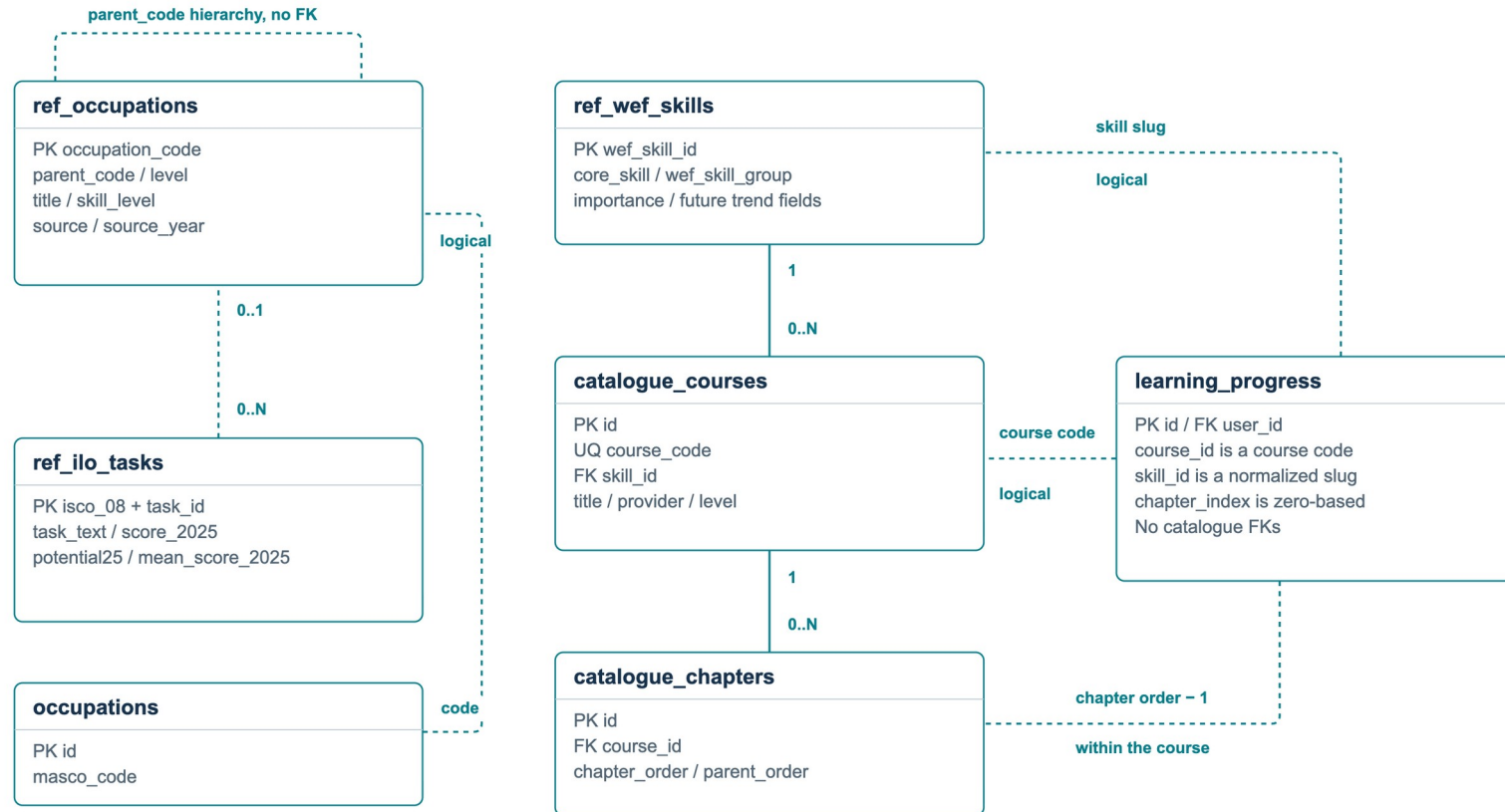
Current backend work and planning model definitions. A model or registered API does not by itself prove that every current page uses it. The task-capability association is shown as a defined model whose explicit runtime write path was not found. [S7]

Appendix C. Reference data and catalogue

AI-WREVOLUSI / ITERATION 2

Reference data and course catalogue

Active SQL reference tables and ORM catalogue at aa4f8fb • Logical associations do not imply database constraints



catalogue_chapters.parent_order is an optional within-course hierarchy value, not a declared self FK.

PK = primary key FK = declared foreign key UQ = unique Solid = declared FK Dotted = logical association only
UQ catalogue_chapters = course_id + chapter_order. Reference table columns are declared in db/schema.sql.

Current reference and catalogue model. Code and chapter-index associations are distinct from declared foreign keys. Source data preparation and import take place outside normal browser requests. [S7]

Appendix D. Separate legacy business schema

AI-WREVOLUSI / ITERATION 2

Legacy SQL model

Eight business tables in db/schema.sql at aa4f8fb • No current backend consumer found • Existence in live database unverified



task_assessments has no unique profile_task_id constraint. The SQL permits zero or many assessments per task.

PK = primary key FK = declared foreign key UQ = unique Solid = declared FK Dotted = logical association only
Grey entities are historical SQL declarations, not claimed active application storage. This is not a deletion recommendation.

Older business tables retained in db/schema.sql. They are separate from the current account workspace model. Their presence in the SQL file does not prove current backend use or live database contents. They are included for historical comparison, not as a proposed migration. [S2, S7]